EAST WEST UNIVERSITY

# Student Wellness Management System

## Submitted to

K. M. Safin Kamal (KMSK)

**Lecturer**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

## SUBMITTED BY:


| Name | ID |
|---|---|
| Md. Rakibul Hasan | 2025-2-50-009 |
| Anjum Bin Ahsan | 2025-2-60-303 |
| Md. Zahid Hossen Majumder | 2025-2-60-377 |

Department of CSE


Date: 4th September 2025

# Student Wellness Management System

A C-based console program to manage student mood monitoring, appointments, and faculty/consultant recommendations, supporting basic administrative functions in a file-based environment. This Report will guide the process of this program and its respective features.

**This application is designed to track student wellness through:**

- Mood entries,
- Faculty recommendations
- Consultant appointments,
- Administration of faculty and consultants.

*All data is stored in text files for simple persistent storage.*

## Quick Overview of the Program Functionality

### 1. Admin

- Login required (default user/pass hardcoded).
- Can add, view, and manage:
    - Registered students
    - Faculty
    - Consultants

### 2. Student

- Registers and logs in.
- Can:
    - Enter daily mood (triggers auto-consultation if mood is low)
    - View mood reports (today/weekly/monthly/3 months)
    - See recommendations and schedule appointments with consultants
    - See upcoming appointments

## 3. Faculty

- Logs in and manages students by course/section.
- Can:
    - See average moods for their section
    - Identify and recommend “vulnerable” students for consultation
    - View recommendation statuses

## 4. Consultant

- Logs in
- Can:
    - See pending recommendations for consultation
    - Schedule appointments for students based on recommendations
    - View one's own appointment schedule

## Functionality Breakdown

## Main features:

- Header System with Divided Content
- Registration & Login
- Admin, Student, Faculty (teacher), Consultant.
- Mood tracking
- Students enter a mood score (1–10) and optional notes.
- Low moods (<5) trigger an automatic consultation recommendation.
- Reporting
- Students can view mood history for various time frames.
- Recommendation
- Faculty can recommend students for consultation based on mood averages or trends.
- Consultants view and process recommendations.
- Appointments
- Consultants schedule appointments with students.
- Appointments managed by both students and consultants

# Program Functionality Table of Contents



## Use of Libraries

The basic libraries used to equip the program with basic but powerful tools for file management, string and memory operations, and date/time handling. Each is essential for the sort of data-heavy, user-interactive code seen in your SWMS (Student Wellness Management System) project

| Include | Used For | Typical Functions Used |
|---|---|---|
| `stdio.h` | Input/Output, file handling | `printf, scanf, FILE, fgets` |
| `stdlib.h` | Memory allocation, process control, conversions | `malloc, free, exit, atoi` |
| `string.h` | String manipulation | `strcpy, strcmp, strlen` |
| `time.h` | Date and time operations | `time, localtime, struct tm` |

## Use of Define Variables

The Define preprocessor directive is used to create named constants and macros for values that are used multiple times throughout your program. This technique makes the code easier to read, maintain, and change, since you only need to update the value in one place if it needs adjustment.

**Why is #define Used?**

- **Convenience:** You only need to change the value once, instead of updating the same number or string throughout your code.
- **Clarity:** Instead of magic numbers or strings, meaningful names tell readers what a value means (e.g., `max_un` is clearer than just 30).
- Maintenance: Easier to update the maximum allowed username length, filename, etc.

| Macro | Value | Purpose/Explanation |
|---|---|---|
| **max_un** | 30 | The maximum length (in characters) for usernames or student IDs. |
| **max_pass** | 30 | The maximum length for passwords. |
| **max_course** | 20 | The maximum length for a course string. |
| **max_sec** | 16 | The maximum length for a section string. |
| **max_note** | 260 | The maximum length for a mood note (text description attached to a mood entry). |
| **maxcpt** | 10 | Not directly described, but likely represents a maximum count, such as max courses per student, etc. |
| **stdfile** | "students.txt" | The filename for the student data file. |
| **teachfile** | "teachers.txt" | The filename for the teacher/faculty data file. |
| **consultfile** | "consultants.txt" | The filename for the consultants data file. |
| **moodentryfile** | "mood_entries.txt" | The filename for student mood entries. |
| **recommendfile** | "recommendations.txt" | The filename for recommendations generated by faculty or automatically. |
| **apptfile** | "appointments.txt" | The filename for storing appointment information (student-consultant appointments). |
| **clrcmd** | "cls"(Windows),"clear"(others) | Sets the correct console clear command depending on OS **(forclrscrn()function).** |

## Use of Headers

The **`common.h`** header file serves as the central interface and shared resource for the entire project. It contains:

- **Constants and macros**: Max sizes for strings, filenames for data storage.
- **Shared data structures**: Definitions for `Student, Teacher, Consultant, MoodEntry, Recommendation, and Appointment`.
- **Platform-specific macros**: For clearing screen command `(clrcmd)` on Windows vs. Unix systems.
- **Utility function prototypes:** Common functions used by multiple modules `(e.g., clrscrn(), paucon()).`
- **Function prototypes for each module:** Declares all functions that main and other modules will call, grouped by user roles `(Student, Faculty, Consultant, Admin).`

This header allows consistent use of types, constants, and functions across all source files, preventing duplicate declarations and simplifying maintenance.

1. **Inclusion in Source Files**

All module files **(main.c, admin.c, consultant.c, faculty.c, student.c)** include **common.h** at the top via: `#include "common.h”`

This grants them access to all shared types, constants, and functions declared in the header

2. **Function Prototypes as Contracts**

**`common.h`** declares prototypes for all functions implemented in the modules. For example:

```
// Student functions
void regstd();
int logstd(char std_id[]);
void stdmenu(char std_id[]);
// Faculty functions
int logfac(char id[], Teacher *t);
void teachmenu(Teacher t);
// Admin functions
int login_admin();
void admin_menu();
// Consultant functions
int logconst(char id[], Consultant *c);
void constmenu(Consultant c);
```

This allows **`main.c`** to know these function signatures, so it can call these functions without requiring their implementation code in scope.

3. **Main Program Flow in main.c**

- **`main.c`** acts as the entry point.
- It presents a main menu that prompts the user to select a role or registration.
- Based on input, it calls appropriate module functions declared via **`common.h`**:
- **`regstd()`** for student registration.
- **`logstd()`** and then **`stdmenu()`** for student login/session.
- **`logfac()`** and **`teachmenu()`** for faculty
- **`logconst()`** and **`constmenu()`** for consultants.
- **`login_admin()`** and **`admin_menu()`** for admin.

This modular flow keeps **main.c** clean and focused on navigation and role management, while actual logic resides in respective modules.

4. **Sharing Data Types Across Modules**

- Struct definitions like Student, Teacher are centralized in **`common.h`**.
- Modules pass these structs across functions, enabling inter-module data sharing with consistent formats.
- For example, **`logfac()`** fills a Teacher struct passed by reference, which is then used by **`teachmenu().`**

5. **Utilities and Platform Compatibility**

- Utilities like **`clrscrn()`** (clear screen) and **`paucon()`** (pause console) are declared in **`common.h`** and implemented in one or more modules (likely **`student.c`** here based on your files).
- The macro **`clrcmd`** adapts screen clearing command based on OS.

This ensures consistent user interface behavior across the program.

## Data Structures

### 1. Student

Represents a student user.

```
typedef struct {
    char std_id[max_un];
    char pass[max_pass];
    char course[max_course];
    char sec[max_sec];
} Student;
```

- **std_id**: Unique student ID.
- **pass**: Password (plaintext, for demo purposes).
- **course**: Registered course code/name.
- **sec**: Class section.

### 2. Teacher

Represents a faculty member.

```
typedef struct {
    char username[max_un];
    char pass[max_pass];
    char course[max_course];
    char sec[max_sec];
} Teacher;
```

- Same field logic as Student, using `username` instead of **`std_id`**.

### 3. Consultant

Represents a consultant.

```
typedef struct {
    char username[max_un];
    char pass[max_pass];
} Consultant;
```

### 4. Mood Entry

Tracks a student's daily mood submission.

```
typedef struct {
    char std_id[max_un];
    char date[11]; // YYYY-MM-DD
    int mood;      // Mood scale, e.g., 1-10
    char note[max_note];
} MoodEntry;
```

### 5. Recommendation

A faculty/automatic suggestion that a student should seek a consultation (based on mood data).

```
typedef struct {
    long id;
    char std_id[max_un];
    char faculty_username[max_un];
    int mood_lvl;
    char type;    // consult/break
    char status;  // pending/accepted/rejected/scheduled
} Recommendation;
```

### 6. Appointment

Scheduled consultation between student and consultant.

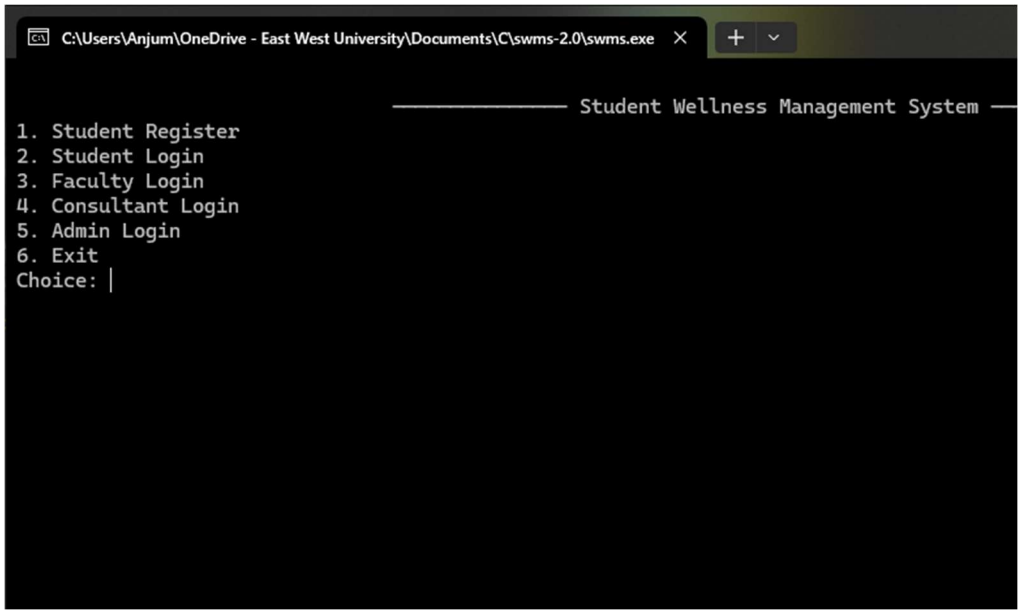
```
typedef struct {
    long id;
    char student_id[max_un];
    char consultant_un[max_un];
    char faculty_un; // "-" if not used
    char date;
    char time;
    char status;
} Appointment;
```

## File Management

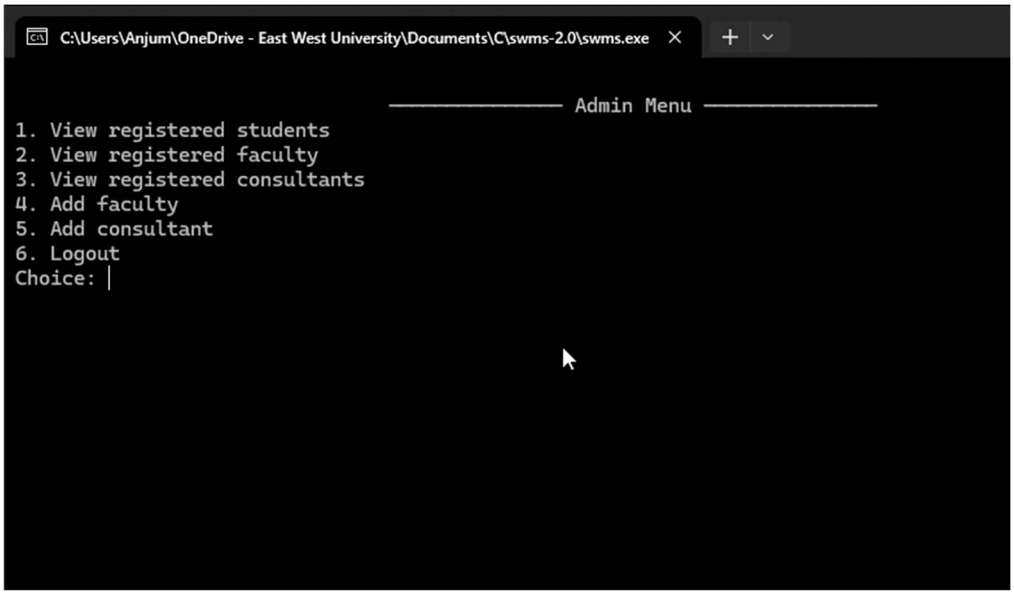
All data is stored in the following files (new line per record):

| File | Content |
|---|---|
| students.txt | Student account info |
| teachers.txt | Faculty account info |
| consultants.txt | Consultant account info |
| mood_entries.txt | Daily mood entries |
| recommendations.txt | Faculty/auto recommendations |
| appointments.txt | Consultant appointments |

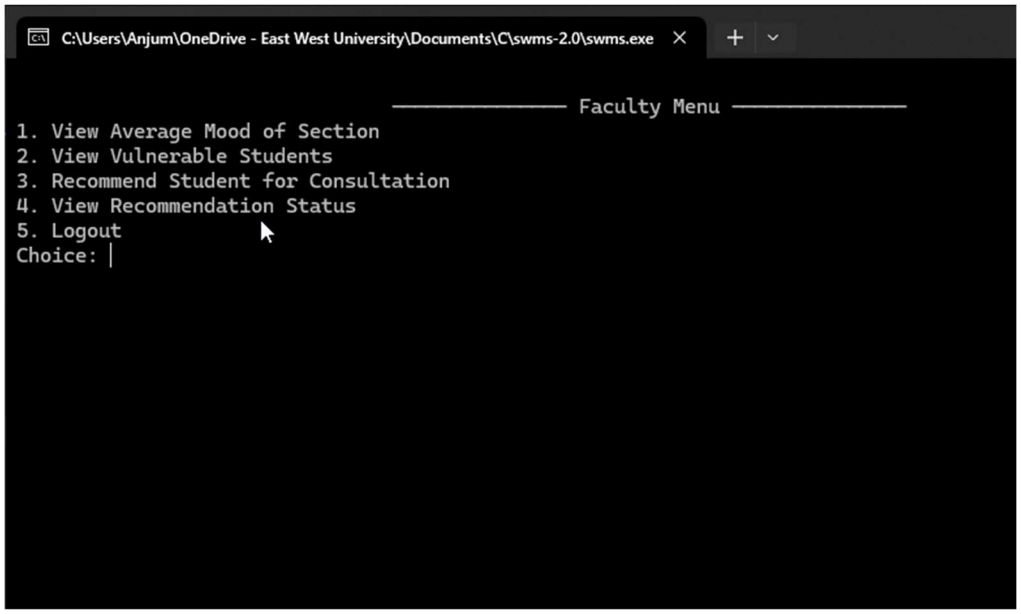
## Screenshots of Menu Screens & Prompts



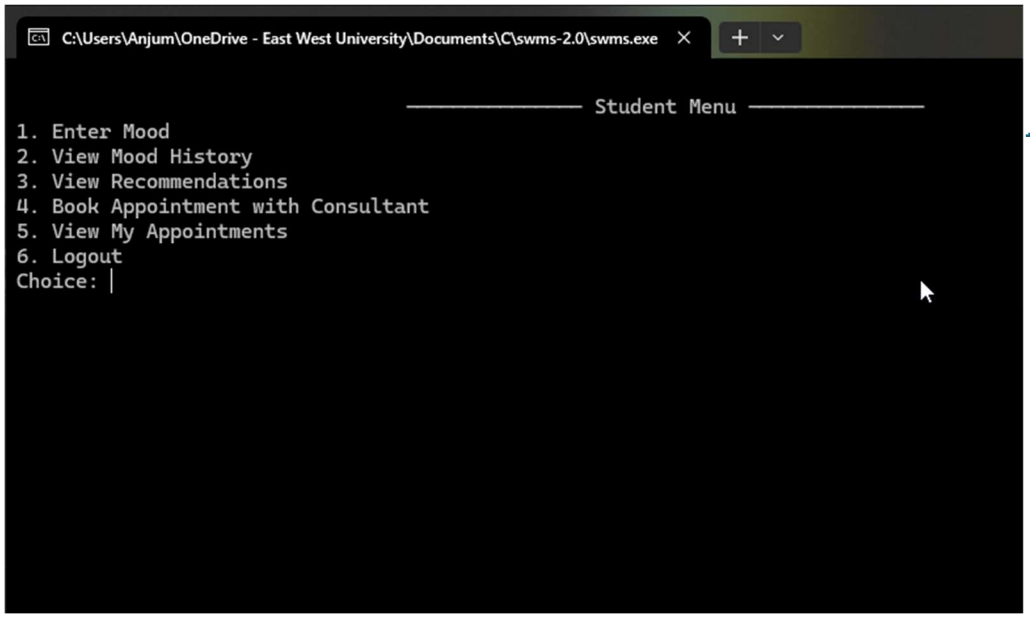

This is the main menu of the program containing the Student, Faculty, Consultant and admin sections respectively



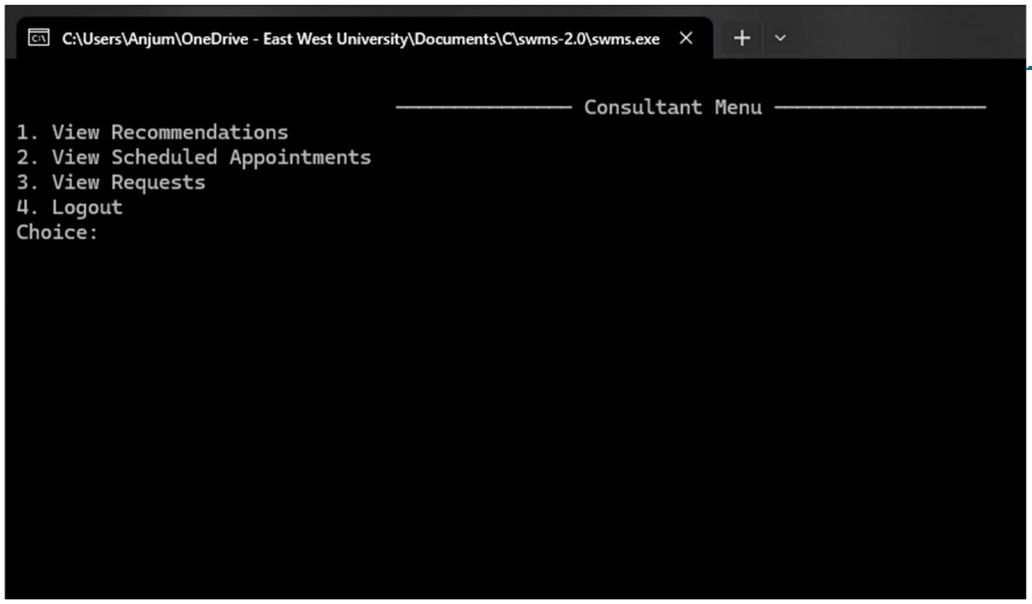

This is the admin menu of the program containing administrative controls and data



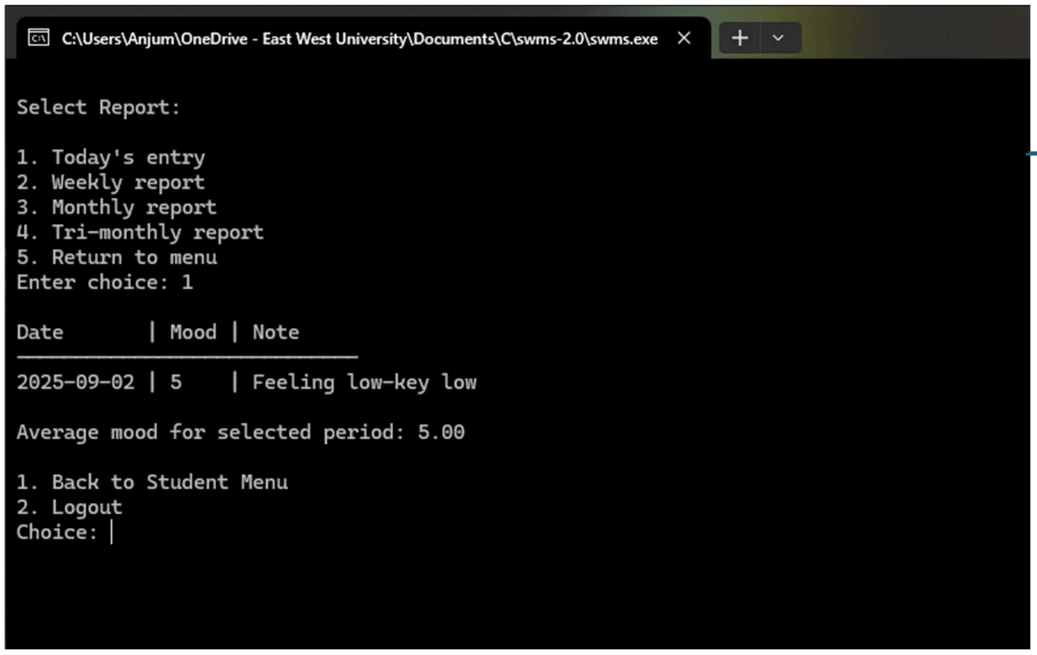

This is the faculty menu of the faculty related data and controls



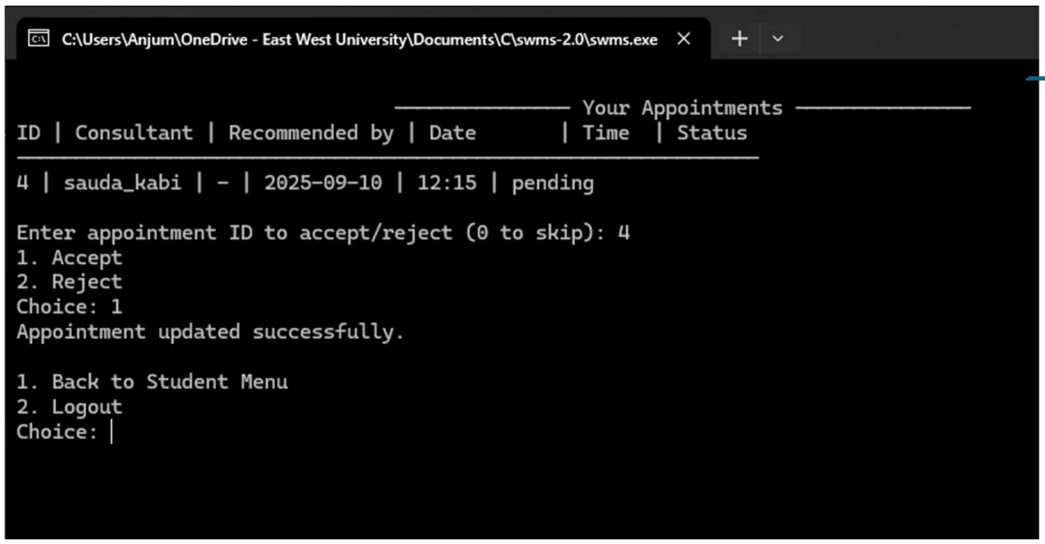

This is the student menu of the student related functions and data



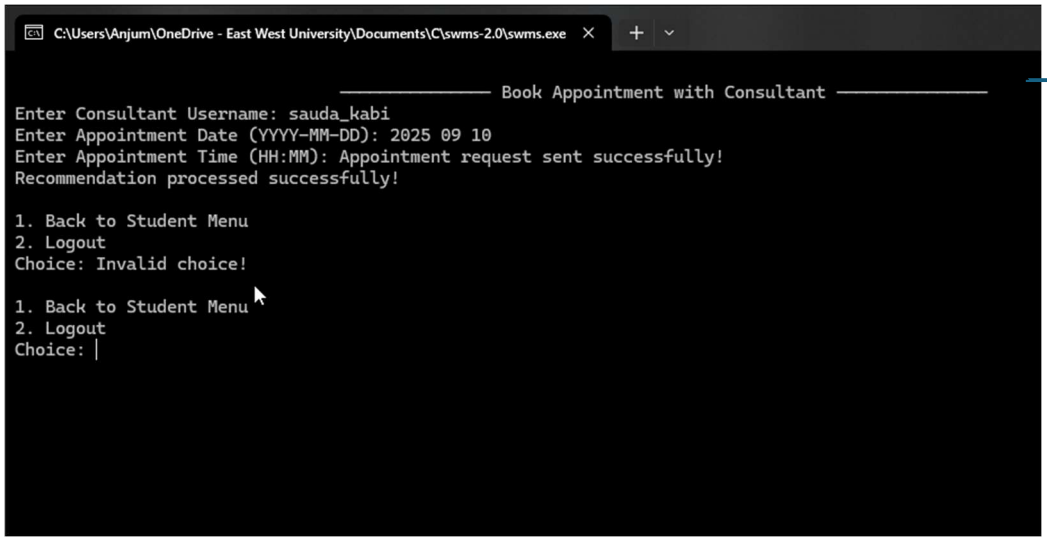

This is the consultant menu for consults and their related functions



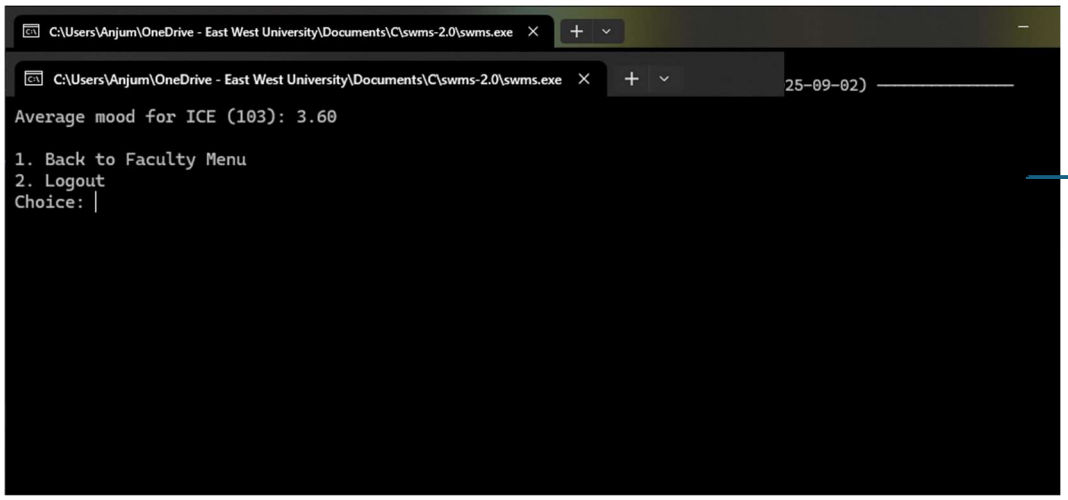

Report Entry section for the students to see their mood for respective time lines based on their previous input



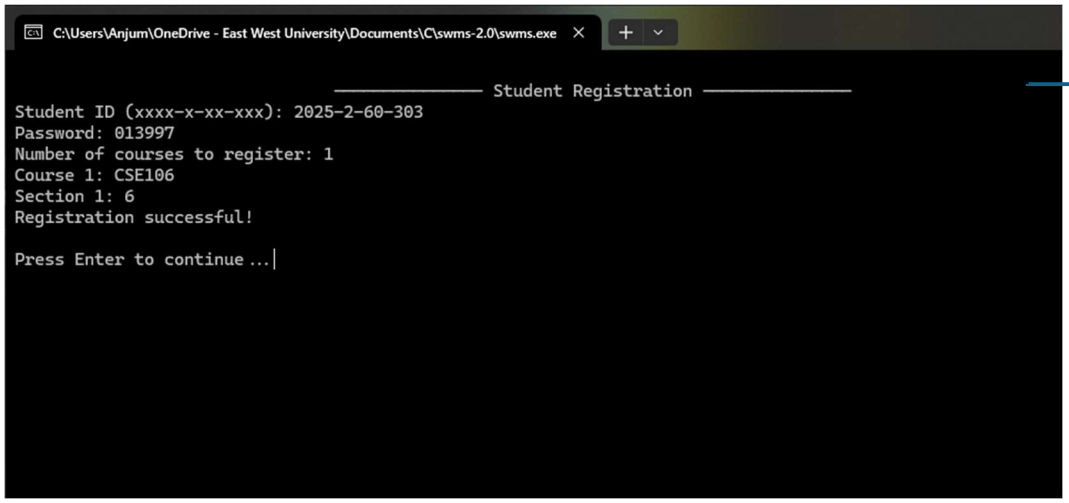

Appointment section where students can see their appointments given by their consultants and if they



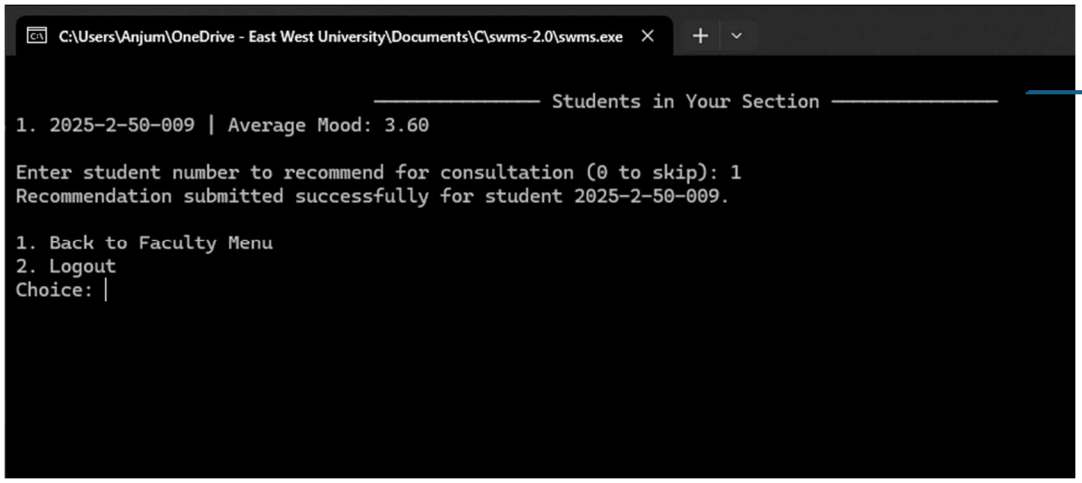

Booking system for appointments where students choose their desired consultant ant respective data



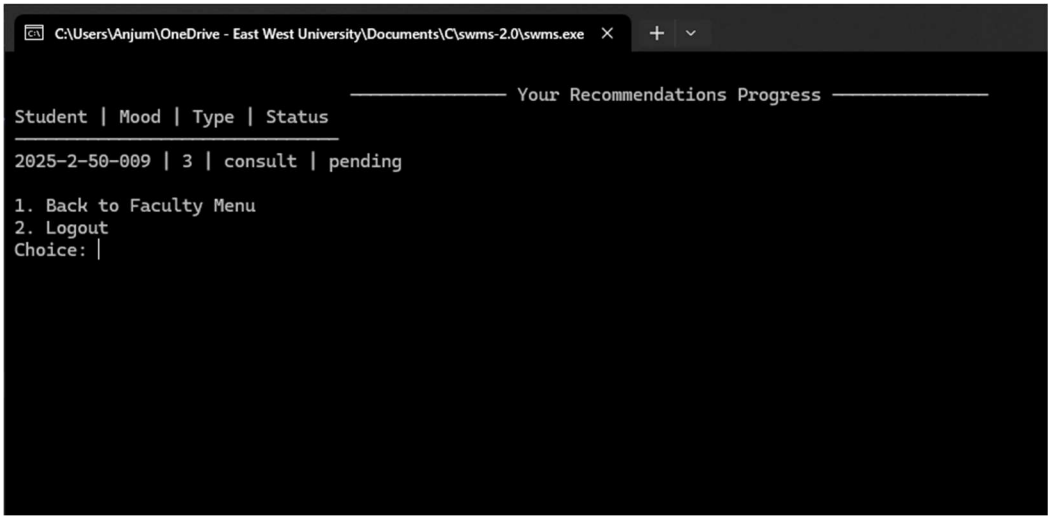

Average mood a student of each faculty respective seen by faculty



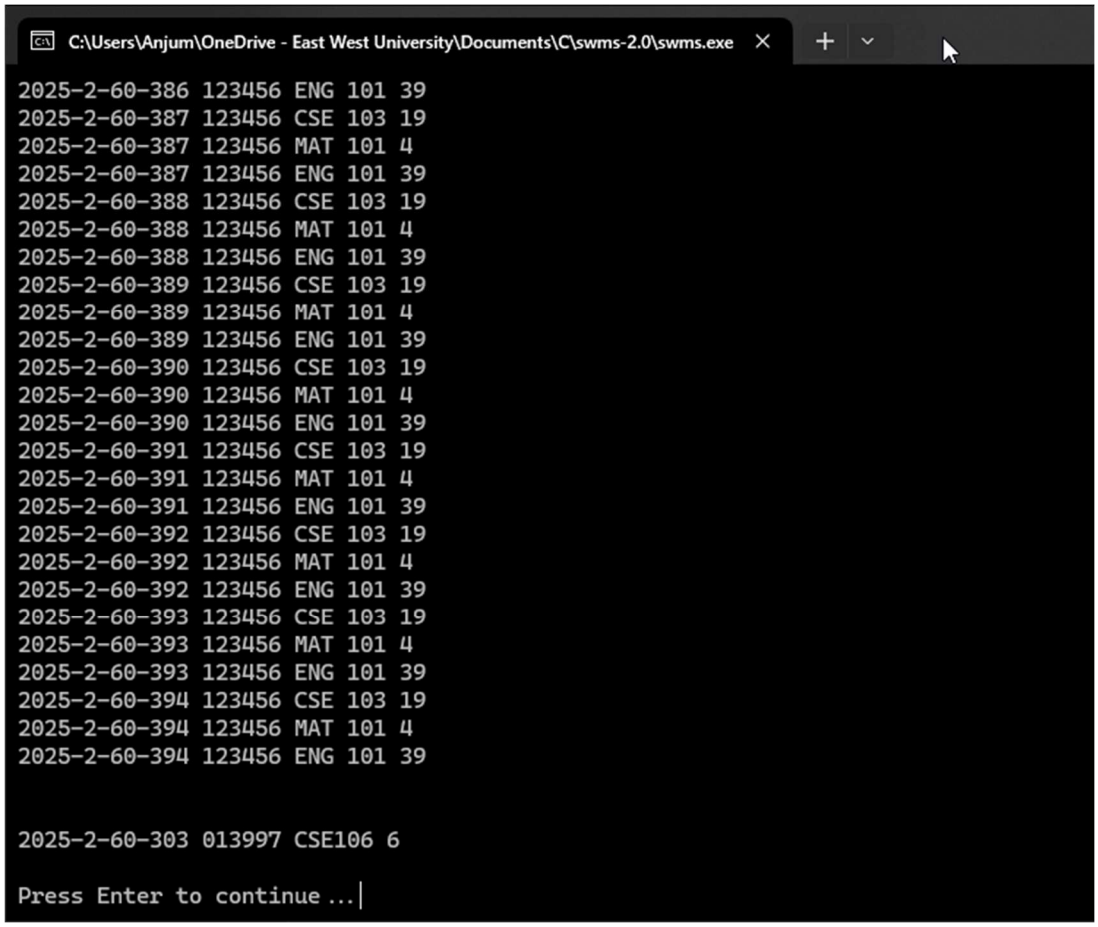

Student registration system for newly students to make their SWMS account



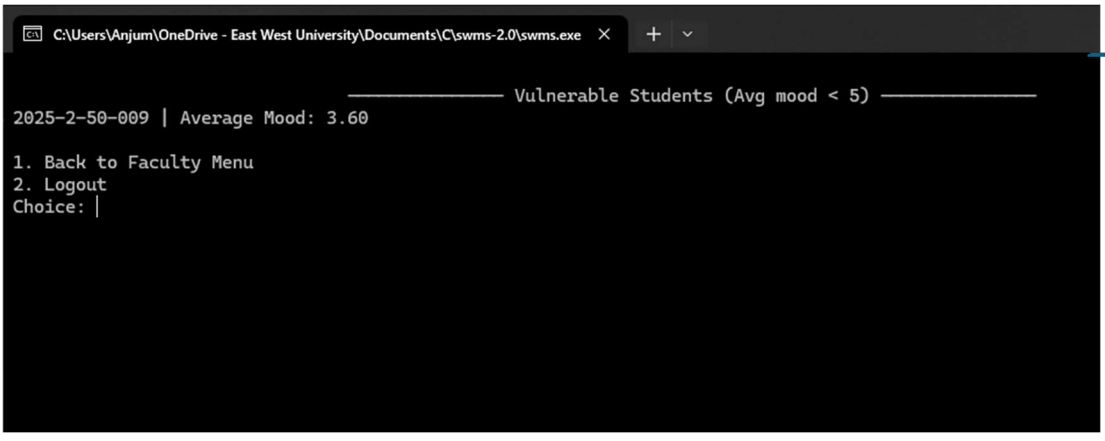

Students and their mood list for respective faculty in faculty section



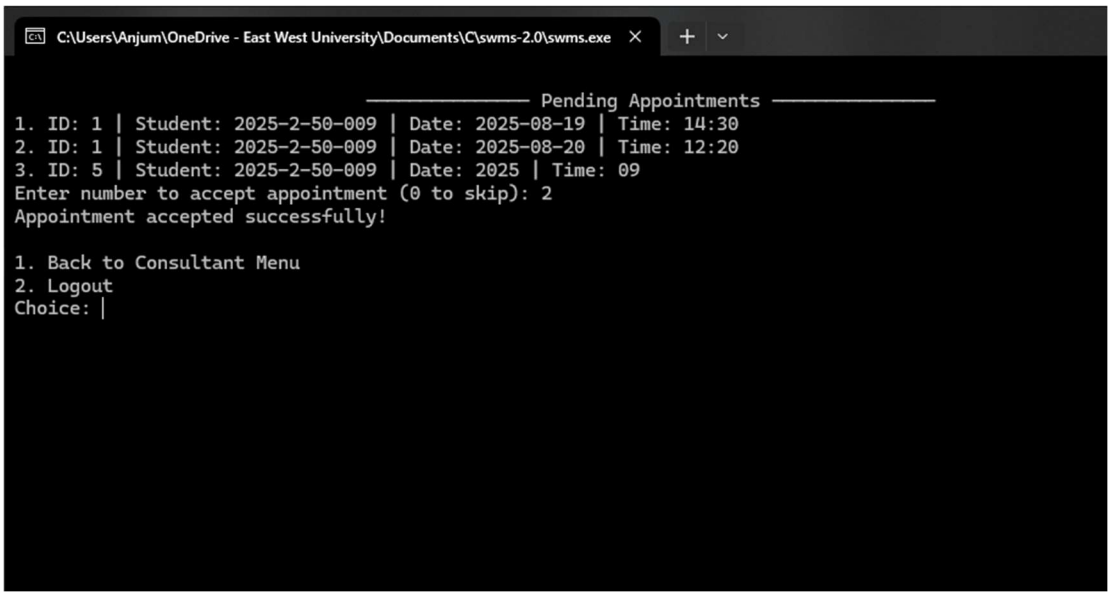

Faculty recommendation system where faculty recommends based on the mood of the student



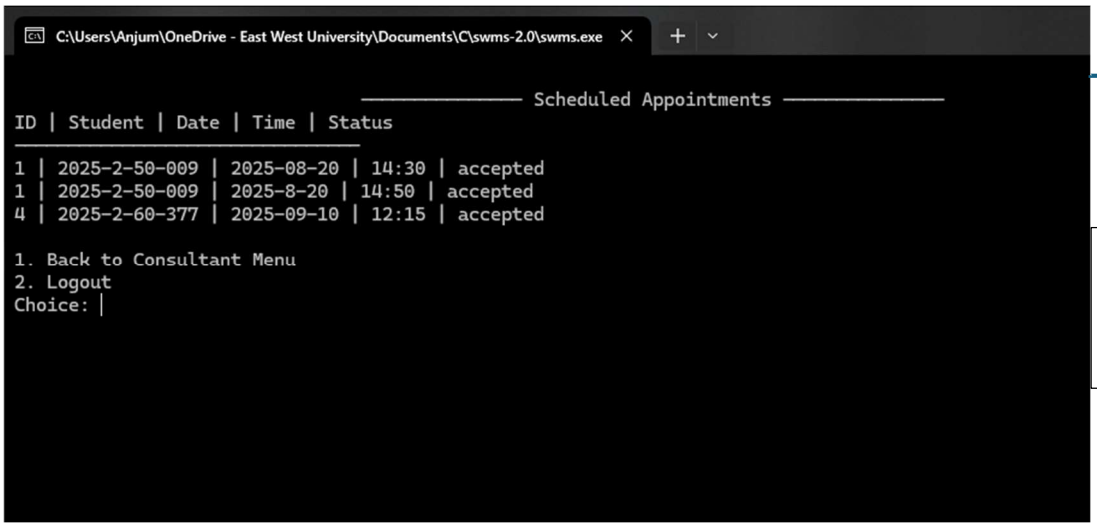

Registered student list of respective faculty




List of vulnerable students which mood is less than 5 of faculty respective




Pending appointments for consultants for respective consults




Scheduled appointments for consultants for respective consults

## Objective of this program

- To provide an effective, user-friendly, and role-based platform for tracking, managing, and improving student mental wellness within an academic institution.
- Allowing students to log their daily moods
- Enabling faculty to monitor collective well-being and recommend vulnerable students for consultation, and giving consultants a streamlined method to manage and schedule support appointments
- Provide administrators a tool to manage users and maintain system integrity

## Data Flow & Example Workflow

### *Student Mood Entry → (Possible) Consultation Path*

- Student logs in and enters mood.
- If mood < 5: automatic Recommendation record is created.
- Faculty reviews class mood data, can recommend any student for consultation.
- Consultant reviews pending recommendations.
- Consultant schedules/accepts an appointment with the student.
- Student is notified of the appointment.

### *Example Function Call Sequence*

- Student registration: regstd() → writes to students.txt.
- Mood entry: entermood() → saves to mood_entries.txt; recommendation if needed.
- Faculty checks section average: fvam(Teacher t) → reads mood_entries.txt for section's students.
- Consultant schedules appointment: consvrecomm(Consultant c) → reads recommendations.txt for pending entries, schedules via appointments.txt.

### *Error Handling*

- **File operations:** Errors on opening files print messages; typically return or halt the function.
- **Input validation:** Ensures that the expected data type/format is received.

### *Design Considerations & Limitations*

- Plaintext password storage, for demonstration simplicity only.
- All storage is file-based (no database), loaded into memory as arrays.
- Scalability is limited by hardcoded array sizes (e.g., 100 records per structure).
- No concurrency handling for file IO.
- Console-based, English-only prompts.

## Important Functions (with explanations)

| Return Type | Function Name | Parameters | Explanation |
|---|---|---|---|
| `void` | `clrscrn` | `none` | Clears the console screenbased on the operating system. Useful for menu navigation and better readability. |

| | | | |
|---|---|---|---|
| void | paucon | none | Pause for user input:Prompts "Press Enter to continue..." before resuming; creates a user-friendly pause. |
| int | login_admin | none | Authenticates admin loginby checking entered username/password against hardcoded admin credentials. |
| void | adminaddfac | none | Admin adds new faculty:Prompts for credentials and appends them to the faculty file. |
| void | adminaddconst | none | Admin adds new consultant:Prompts for credentials and appends them to the consultant file. |
| void | admin_menu | none | Admin menu:Displays options (view/add users) and dispatches selected admin actions. |
| void | regstd | none | Student registration:Gets student info, asks for course(s), and saves to the students file. |
| int | spfm | none | Not shown in supplied content.Typically would return (possibly from a menu) to main student UI. |
| int | view_report | char std_id[] | Shows mood reports:Student views their own mood history for selected time period. |
| int | loginstudent | char std_id[], Student *s | Student login:Authenticates and loads student's data into structure. |
| void | stdmenu | char std_id[] | Main menu for students:Presents student actions and manages interaction loops. |
| int | srvr | char std_id[] | Student receives/view recommendationsand can respond/book appointments. |
| int | sba | char std_id[] | Student book appointment:Handles logic for booking a consultation based on recommendation status. |
| int | sva | char std_id[] | Student view appointments:Displays studentâ€™s own booked appointments. |
| int | fvam | Teacher t | Faculty views average moodfor their assigned section and course. |
| int | facvulstd | Teacher t | Faculty view vulnerable students:Finds students with low average mood for recommendations. |
| int | facvrecommsta | Teacher t | Faculty view recommendation status:Shows statuses of their submitted recommendations. |
| void | teachmenu | Teacher t | Main menu for faculty:Handles all faculty actions from a central UI. |

| | | | |
|---|---|---|---|
| `int` | `login_teacher` | `char username[], Teacher *t` | Faculty login:Authenticates teacher and fills in given Teacher structure. |
| `int` | `facrecconst` | `Teacher t` | Faculty recommend a student:Prompts to select/submit a recommendation for consultation. |
| `int` | `logconst` | `char id[], Consultant *c` | Consultant login:Authenticates consultant and loads credentials into struct. |
| `int` | `consvrecomm` | `Consultant c` | Consultant view/pick recommendationsand schedule appointments. |
| `int` | `consvsche` | `Consultant c` | Consultant view scheduled appointments:Shows accepted appointments for consultant. |

## Detailed Function Explanations

### *General Utilities*

- **`void clrscrn(void)`**
  - Purpose: Clears the console after switching menus.
  - How: Calls system("cls") or system("clear"), depending on OS.

- **`void paucon(void)`**
  - Purpose: Waits for user to press enter as a pause after completing some work—improves navigation for users.

### *Admin-Related*

- **`int login_admin(void)`**
  - Purpose: Takes admin username and password, checks if they match hardcoded credentials ("admin_swms" / "swmsewu2025").
  - Returns: 1 for success, 0 for failure.

- **`void adminaddfac(void)`**
  - Purpose: Prompts user for new faculty username, password, course, section, and writes them to teachers.txt.

- **`void adminaddconst(void)`**
  - Purpose: Prompts user for new consultant username and password, writes them to consultants.txt.

- **`void admin_menu(void)`**
    - Purpose: Displays admin menu and lets admin view/add students, faculty, and consultants; dispatches tasks based on selection.

***Student-Related***

- **`void regstd(void)`**
    - Purpose: Registers a new student; prompts for ID, password, number of courses, then collects course and section info.

- **`int spfm(void)`**
    - Purpose: (Definition not shown, but presumably a helper that returns to student menu or previous function.)

- **`void stdmenu(char std_id[])`**
    - Purpose: Main command loop for student users, showing all student actions.

- **`int loginstudent(char std_id[], Student *s)`**
    - Purpose: Authenticates student by reading from students file and matching ID/password. Loads details into provided structure.

- **`int view_report(char std_id[])`**
    - Purpose: Lets students choose a period and then displays mood history (date, mood, note) for that window.

- **`int srvr(char std_id[])`**
    - Purpose: Student views recommendations, and can take action (e.g., book appointments).

- **`int sba(char std_id[])`**
    - Purpose: Student books an appointment with a consultant after receiving a recommendation.
- **`int sva(char std_id[])`**
    - Purpose: Displays a list of all appointments for the logged-in student.

***Faculty/Teacher-Related***

- **`int login_teacher(char username[], Teacher *t)`**
    - Purpose: Authenticates faculty user and loads details into supplied structure.

- **`void teachmenu(Teacher t)`**
    - Purpose: Main menu/loop for faculty actions, e.g., view moods, recommend students, view statuses.

- **`int fvam(Teacher t)`**
    - Purpose: Calculates and displays the average mood for all students in the teacher's course and section.

- **`int facvulstd(Teacher t)`**
    - Purpose: Identifies students with low average moods for possible faculty intervention/recommendation.

- **`int facrecconst(Teacher t)`**
    - Purpose: Faculty recommends a student for consultation. (Implementation not shown, but called in menu.)

- **`int facvrecommsta(Teacher t)`**
    - Purpose: Shows all recommendations made by the current faculty member and their status.

***Consultant-Related***

- **`int logconst(char id[], Consultant *c)`**
    - Purpose: Authenticates a consultant by reading the respective file and matching entered credentials.

- **`int consvrecomm(Consultant c)`**
    - Purpose: Consultant loads all pending recommendations and can schedule new appointments.

- **`int consvsche(Consultant c)`**
    - Purpose: Consultant views their schedule of already accepted appointments.

## Conclusion

The Student Mental Welfare System successfully bridges the communication gap that students, faculties, consultants, and administrators face regarding mental health and well-being reporting. The modular design of this program is intended to enable effective user input and usage by the user. The project, with its features for mood tracking, reporting, recommendations, and appointment management, promotes early intervention and support for students who are undergoing severe mental issues that cause trouble in their daily lives. The system introducesitself as a probable future scalable digital wellness solution in educational environments and can be expanded further to incorporate additional security measures and features.

**Flow Chart**

**Note:** Each Path of C file is color coded. (→)Arrow is read-only and (←->) Arrow is Read/Write